

Universidad de los Andes

Departamento de Ingeniería de Sistemas y Computación



Taller 2

ISIS1611 – Inteligencia Artificial

Semestre 20261

Integrantes:

Mateo Rincon – 202221402

Silvana Echeverry – 202310470

Sofía Morato – 202321074

Juan Esteban Jiménez Benavides 201922487

Tabla de contenido

Punto 4a	3
1. Backtracking básico	3
2. Forward Checking	4
3. Backtracking con AC-3.....	6
4. Backtracking con MRV + LCV + Forward Checking.....	9
5. Reducción del espacio de búsqueda vs. búsqueda exhaustiva.....	11
6. Resumen de complejidades	13
Punto 4b	13
b) Búsqueda adversaria (6%).....	13
Complejidad en tiempo y espacio de Minimax	13
Llamada inicial en get_action	13
Dentro de la función recursiva minimax(state, depth, agent_index).....	14
pesimista con cazadores no óptimos	16
Función de evaluación: características, pesos y justificación	16
Comparación de tasas de victoria en diversos layouts	16
Nodos adicionales que poda AlfaBeta (mejor y peor caso)	17
c) Reflexión sobre la coconstrucción con IA (3%)	17

Punto 4a

Para el análisis de complejidad temporal y espacial se utilizan las siguientes variables:

Variable	Definición
n	Número de variables del CSP (puntos de entrega a asignar)
d	Tamaño máximo del dominio (número de drones disponibles)
c	Número de restricciones binarias activas entre pares de variables

1. Backtracking básico

Se analizan las dos funciones: `backtracking_search()` es el punto de entrada y `backtracking()` es la función recursiva principal.

Complejidad en tiempo (Notación O)

`backtracking_search()`:

Código	Complejidad	Explicación
<code>assignment = {}</code>	$O(1)$	Inicialización de un diccionario vacío.
<code>return backtracking(csp, assignment)</code>	$O(1)$	Llamada inicial a la función recursiva. El costo real está en <code>backtracking()</code> .

`backtracking()`:

Código	Complejidad	Explicación
<code>if csp.is_complete(assignment): return assignment.copy()</code>	$O(n)$	<code>is_complete</code> compara <code>len(assignment)</code> con <code>len(variables)</code> : $O(1)$. <code>assignment.copy()</code> copia n entradas: $O(n)$.
<code>var = csp.get_unassigned_variables(assignment)[0]</code>	$O(n)$	Recorre las n variables filtrando las no asignadas. Acceder al índice [0] es $O(1)$.
<code>for value in csp.domains[var]:</code>	$O(d)$	En el peor caso itera sobre los d valores del dominio de var.
<code>if csp.is_consistent(var, value, assignment):</code>	$O(n)$	Verifica capacidad, batería y ventana de tiempo recorriendo las n asignaciones actuales.
<code>csp.assign(var, value, assignment)</code>	$O(1)$	Inserta un par clave-valor en el diccionario <code>assignment</code> .

<pre>result = backtracking(csp, assignment) if result is not None: return result</pre>	$O(d^n)$	Llamada recursiva. El árbol tiene hasta d ramas por nivel y profundidad n , generando d^n nodos en el peor caso.
<pre>csp.unassign(var, assignment)</pre>	$O(1)$	Elimina la clave <code>var</code> del diccionario.
<pre>return None</pre>	$O(1)$	Retorno cuando se agotan todos los valores del dominio sin solución.

Cada nodo del árbol realiza $O(d \cdot n)$ trabajo (d valores $\times O(n)$ por `is_consistent`). Con d^n nodos en el peor caso, la complejidad temporal total es $O(n \cdot d^n)$, simplificada a $O(d^n)$ dado que n es constante para un problema fijo.

Complejidad en espacio (Notación O)

Estructura	Espacio	Justificación
<code>assignment = {}</code>	$O(n)$	Diccionario con como máximo n pares variable-valor.
Pila de llamadas recursivas	$O(n)$	La profundidad máxima de la recursión es n : una llamada activa por variable asignada.

La complejidad espacial del backtracking básico es $O(n)$. No genera copias de dominio ni guarda estado adicional entre niveles.

2. Forward Checking

Se analizan las tres funciones: `backtracking_fc()` es el punto de entrada, `forward_check()` realiza la propagación, y `backtrack_fc()` es el bucle recursivo principal.

Complejidad en tiempo (Notación O)

`backtracking_fc()`:

Código	Complejidad	Explicación
<pre>assignment = {}</pre>	$O(1)$	Inicialización del diccionario de asignación vacío.
<pre>domains = {var: list(values) for var, values in csp.domains.items()}</pre>	$O(n \cdot d)$	Crea una copia local de los dominios: n variables, cada una con hasta d valores. Esta copia es la que se modifica durante la búsqueda.
<pre>return backtrack_fc(csp, assignment, domains)</pre>	$O(1)$	Llamada inicial. El costo está en <code>backtrack_fc()</code> .

`forward_check()`:

Código	Complejidad	Explicación
<pre>for neighbor in csp.get_neighbors(var):</pre>	$O(n)$	Itera sobre los $n-1$ vecinos de <code>var</code> . <code>get_neighbors()</code> retorna todas las variables excepto <code>var</code> .

if neighbor in assignment: continue	O(1)	Lookup O(1) en diccionario. Saltea vecinos ya asignados.
valid_values = [] for value in domains[neighbor]:	O(d)	Itera sobre el dominio residual del vecino, que tiene hasta d valores.
if csp.is_consistent(neighbor, value, assignment):	O(n)	Chequeo de consistencia: verifica capacidad, batería y ventana de tiempo contra las n asignaciones actuales.
domains[neighbor] = valid_values	O(d)	Reemplaza el dominio del vecino con la lista de valores válidos.
if len(domains[neighbor]) == 0: return False	O(1)	Verifica si el dominio quedó vacío: fallo detectado anticipadamente.
return True	O(1)	Todos los vecinos tienen al menos un valor válido.

Costo total de forward_check por cada asignación: $O(n \cdot d \cdot n) = O(n^2 \cdot d)$: n vecinos × d valores × O(n) por is_consistent.

backtrack_fc():

Código	Complejidad	Explicación
if csp.is_complete(assignment): return assignment.copy()	O(n)	Misma verificación que en backtracking básico.
var = csp.get_unassigned_variables(assignment)[0]	O(n)	Recorre las n variables para encontrar las no asignadas.
for value in domains[var]:	O(d)	Itera sobre el dominio residual de var (ya reducido por propagaciones anteriores).
if csp.is_consistent(var, value, assignment):	O(n)	Verificación de consistencia antes de asignar.
csp.assign(var, value, assignment)	O(1)	Inserta en el diccionario assignment.
saved_domains = {v: list(vals) for v, vals in domains.items()} domains[var] = [value]	O(n · d)	Guarda copia completa de todos los dominios actuales (n variables × d valores) para poder restaurarlos al hacer backtrack. Reduce el dominio de var al valor asignado.
if forward_check(csp, var, assignment, domains):	O(n ² · d)	Propagación a todos los vecinos no asignados (ver análisis anterior).
result = backtrack_fc(csp, assignment, domains) if result is not None: return result	O(d ⁿ)	Llamada recursiva. Árbol de profundidad n con factor de ramificación d.
domains.clear() domains.update(saved_domains) csp.unassign(var, assignment)	O(n · d)	Restaura los dominios guardados y elimina la asignación de var al hacer backtrack.

La complejidad temporal total es $O(n^2 \cdot d \cdot d^n)$ en el peor caso. En la práctica, la poda anticipada de forward_check elimina ramas antes de explorarlas, reduciendo el número real de nodos visitados respecto al backtracking básico.

Complejidad en espacio (Notación O)

Estructura	Espacio	Justificación
assignment = {}	$O(n)$	Diccionario con como máximo n pares variable-valor.
domains = {...} inicial	$O(n \cdot d)$	Copia local de los dominios: n variables × d valores.
saved_domains por nivel	$O(n \cdot d)$	En cada nivel de la recursión se guarda una copia completa de los dominios.
Pila de recursión (n niveles)	$O(n)$	Profundidad máxima n.
Total (n niveles activos)	$O(n^2 \cdot d)$	n niveles simultáneos, cada uno con su copia $O(n \cdot d)$ de saved_domains.

La complejidad espacial de Forward Checking es $O(n^2 \cdot d)$. La estructura dominante son las copias de dominios acumuladas en la pila de recursión.

3. Backtracking con AC-3

La implementación de AC-3 se divide en cuatro funciones: backtracking_ac3() es el punto de entrada, values_compatible() verifica compatibilidad entre dos valores, revise() poda el dominio de una variable, ac3() gestiona la cola de arcos, y backtrack_ac3() es el bucle recursivo.

Complejidad en tiempo (Notación O)

backtracking_ac3():

Código	Complejidad	Explicación
assignment = {}	$O(1)$	Inicialización del diccionario de asignación vacío.
domains = {var: list(values) for var, values in csp.domains.items()}	$O(n \cdot d)$	Copia local de los dominios: n variables × d valores.
return backtrack_ac3(csp, assignment, domains)	$O(1)$	Llamada inicial. El costo está en backtrack_ac3().

values_compatible():

Código	Complejidad	Explicación
if x != y: return True	$O(1)$	Si los drones son distintos no hay restricciones compartidas: retorno inmediato sin cómputo adicional.
drone = csp.drones[x] temp = dict(assignment) temp[x] = x temp[x] = y	$O(n)$	dict(assignment) copia el assignment actual: $O(n)$. Se añaden 2 entradas más: $O(1)$.

<pre>weight = csp._compute_drone_total_weight(x, temp) if weight > drone['capacity']: return False</pre>	O(n)	_compute_drone_total_weight itera sobre temp buscando entradas del drone x: O(n).
<pre>cost = csp._compute_drone_route_cost(x, temp) if cost > drone['battery']: return False return True</pre>	O(n)	_compute_drone_route_cost itera sobre las entregas del drone en temp calculando distancias: O(n).

Costo de valores_compatible: **O(n)**.

revise():

Código	Complejidad	Explicación
values_to_remove = []	O(1)	Inicialización de lista vacía.
for x in domains[xi]:	O(d)	Itera sobre los valores del dominio de xi (hasta d valores).
<pre>supported = any(values_compatible(csp, xi, x, xj, y, assignment) for y in domains[xj])</pre>	O(d · n)	Para cada valor x de xi, evalúa values_compatible contra cada valor y del dominio de xj. Cada llamada cuesta O(n). any() cortocircuita al primer True.
<pre>if not supported: values_to_remove.append(x)</pre>	O(1)	Marca el valor sin soporte para eliminación.
<pre>for x in values_to_remove: domains[xi].remove(x) return len(values_to_remove) > 0</pre>	O(d)	Elimina valores sin soporte. len() es O(1).

Costo total de revise: **O(d² · n)**: d valores de xi × d valores de xj × O(n) por values_compatible.

ac3():

Código	Complejidad	Explicación
while queue:	O(c · d)	El while ejecuta como máximo c·d iteraciones: c arcos y cada arco puede reinsertarse hasta d veces (una por cada valor eliminado de xi).
xi, xj = queue.pop(0)	O(1)	Extrae el primer arco de la cola.
if revise(csp, xi, xj, domains, assignment):	O(d ² · n)	Costo de revise por cada arco procesado (ver análisis anterior).
<pre>if len(domains[xi]) == 0: return False</pre>	O(1)	Dominio vacío: falla inmediatamente.
<pre>for xk in csp.get_neighbors(xi): if xk != xj: queue.append((xk, xi))</pre>	O(n)	Reinserta arcos de los vecinos de xi (hasta n-1 vecinos) en la cola.
return True	O(1)	Cola vacía: consistencia de arco garantizada.

Costo total de ac3: **O(c · d · d² · n) = O(c · d³ · n)**. Como c = O(n²) en este CSP (todas las variables comparten restricciones), la complejidad por invocación es **O(n³ · d³)**.

backtrack_ac3():

Código	Complejidad	Explicación
<pre>if csp.is_complete(assignment): return assignment.copy()</pre>	$O(n)$	Igual que en backtracking básico.
<pre>unassigned = csp.get_unassigned_variables(assignment) var = unassigned[0]</pre>	$O(n)$	Recorre las n variables para obtener las no asignadas.
<pre>for value in list(domains[var]):</pre>	$O(d)$	<code>list()</code> copia el dominio residual para evitar modificarlo durante la iteración: $O(d)$. Luego itera sobre los valores.
<pre> if csp.is_consistent(var, value, assignment):</pre>	$O(n)$	Verificación de consistencia antes de asignar.
<pre> csp.assign(var, value, assignment)</pre>	$O(1)$	Inserta en el diccionario <code>assignment</code> .
<pre> saved_domains = {v: list(vals) for v, vals in domains.items()} domains[var] = [value]</pre>	$O(n \cdot d)$	Copia completa de los dominios actuales para poder restaurarlos en backtrack. Reduce el dominio de <code>var</code> al valor asignado.
<pre> queue = [(neighbor, var) for neighbor in csp.get_neighbors(var) if neighbor not in assignment]</pre>	$O(n)$	Construye la cola inicial de arcos: hasta $n-1$ vecinos no asignados apuntando hacia <code>var</code> .
<pre> if ac3(csp, domains, queue, assignment):</pre>	$O(n^3 \cdot d^3)$	Invocación de AC-3 para propagar consistencia tras la asignación (ver análisis anterior).
<pre> result = backtrack_ac3(csp, assignment, domains) if result is not None: return result</pre>	$O(d^n)$	Llamada recursiva. Árbol de profundidad n con factor de ramificación d .
<pre>domains.clear() domains.update(saved_domains) csp.unassign(var, assignment)</pre>	$O(n \cdot d)$	Restaura los dominios guardados y elimina la asignación de <code>var</code> al hacer backtrack.

La complejidad temporal total es $O(n^3 \cdot d^3 \cdot d^n)$. El término $O(n^3 \cdot d^3)$ es el costo de `ac3` por nodo del árbol. Aunque es mayor que Forward Checking, AC-3 poda más valores de los dominios, reduciendo el número de nodos explorados.

Complejidad en espacio (Notación O)

Estructura	Espacio	Justificación
<code>assignment = {}</code>	$O(n)$	Diccionario con como máximo n pares variable-valor.
<code>domains = {...}</code> inicial	$O(n \cdot d)$	Copia local de los dominios: n variables $\times$ d valores.
Cola de arcos (queue)	$O(c) = O(n^2)$	En el peor caso contiene todos los arcos del CSP.
<code>saved_domains</code> por nivel	$O(n \cdot d)$	Copia completa de dominios en cada nivel de recursión.

Total (n niveles activos)	$O(n^2 \cdot d)$	Dominado por las copias de saved_domains acumuladas en la pila.
---------------------------	------------------	---

La complejidad espacial es $O(n^2 \cdot d)$, igual que Forward Checking. La cola de arcos $O(n^2)$ queda cubierta por las copias de dominio.

4. Backtracking con MRV + LCV + Forward Checking

Se analizan las cuatro funciones: backtrack_mrv_lcv() es el punto de entrada, select_unassigned_variable_mrv() elige la variable, order_domain_values_lcv() ordena los valores, forward_check_mrv() propaga restricciones, y backtrack_mrv_lcv() es el bucle recursivo.

Complejidad en tiempo (Notación O)

backtracking_mrv_lcv():

Código	Complejidad	Explicación
assignment = {}	$O(1)$	Inicialización del diccionario de asignación vacío.
domains = {var: list(values) for var, values in csp.domains.items()}	$O(n \cdot d)$	Copia local de los dominios: n variables × d valores.
return backtrack_mrv_lcv(csp, assignment, domains)	$O(1)$	Llamada inicial. El costo está en backtrack_mrv_lcv().

select_unassigned_variable_mrv():

Código	Complejidad	Explicación
unassigned = csp.get_unassigned_variables(assignment)	$O(n)$	Filtra las n variables para obtener las no asignadas.
return min(unassigned, key=lambda var: (len(domains[var]), -sum(1 for neighbor in csp.get_neighbors(var) if neighbor not in assignment)))	$O(n^2)$	min() recorre hasta n variables. Para cada una, el desempate por grado suma sus vecinos no asignados: $O(n)$ por variable. Total: $O(n^2)$.

Costo de select_unassigned_variable_mrv: $O(n^2)$.

order_domain_values_lcv():

Código	Complejidad	Explicación
valid_values = [value for value in domains[var] if csp.is_consistent(var, value, assignment)]	$O(d \cdot n)$	Filtra los d valores del dominio. Cada is_consistent cuesta $O(n)$.

<pre>return sorted(valid_values, key=lambda value: csp.get_num_conflicts(var, value, assignment))</pre>	$O(n \cdot d^2 \cdot \log d)$	sorted() ordena d valores. La key get_num_conflicts itera sobre vecinos y sus dominios: $O(n \cdot d)$ por valor. Total de la key: $O(n \cdot d \cdot d) = O(n \cdot d^2)$. Con el log d del sort: $O(n \cdot d^2 \cdot \log d)$.
---	-------------------------------	---

Costo de order_domain_values_lcv: $O(n \cdot d^2 \cdot \log d)$.

forward_check_mrv():

Código	Complejidad	Explicación
<pre>for neighbor in csp.get_neighbors(var):</pre>	$O(n)$	Itera sobre los n-1 vecinos de var.
<pre> if neighbor in assignment: continue</pre>	$O(1)$	Lookup en diccionario. Saltea vecinos ya asignados.
<pre> valid_values = [] for value in domains[neighbor]:</pre>	$O(d)$	Itera sobre el dominio residual del vecino.
<pre> if csp.is_consistent(neighbor, value, assignment):</pre>	$O(n)$	Verificación de consistencia contra el assignment actual.
<pre> domains[neighbor] = valid_values if len(domains[neighbor]) == 0: return False return True</pre>	$O(d)$	Actualiza dominio y verifica si quedó vacío.

Costo de forward_check_mrv: $O(n^2 \cdot d)$: n vecinos $\times$ d valores $\times$ $O(n)$ por is_consistent.

backtrack_mrv_lcv():

Código	Complejidad	Explicación
<pre>if csp.is_complete(assignment): return assignment.copy()</pre>	$O(n)$	Igual que en los algoritmos anteriores.
<pre>var = select_unassigned_variable_mrv(csp, assignment, domains)</pre>	$O(n^2)$	Selección de la variable más restringida con desempate por grado.
<pre>for value in order_domain_values_lcv(csp, var, assignment, domains):</pre>	$O(n \cdot d^2 \cdot \log d)$	Ordena los valores del dominio de var por número de conflictos generados en vecinos.
<pre> if csp.is_consistent(var, value, assignment):</pre>	$O(n)$	Verificación de consistencia antes de asignar.
<pre> csp.assign(var, value, assignment)</pre>	$O(1)$	Inserta en el diccionario assignment.
<pre> saved_domains = {v: list(vals) for v, vals in domains.items()} domains[var] = [value]</pre>	$O(n \cdot d)$	Copia completa de los dominios por nivel. Reduce el dominio de var al valor asignado.
<pre> if forward_check_mrv(csp, var, assignment, domains):</pre>	$O(n^2 \cdot d)$	Propagación de restricciones a vecinos no asignados.
<pre> result = backtrack_mrv_lcv(csp, assignment, domains) if result is not None:</pre>	$O(d^n)$	Llamada recursiva. MRV reduce el branching factor efectivo pero el peor caso es d^n .

return result		
domains.clear() domains.update(saved_domains) csp.unassign(var, assignment)	$O(n \cdot d)$	Restaura los dominios y elimina la asignación de var al hacer backtrack.

La complejidad temporal total es $O((n^2 + n \cdot d^2 \cdot \log d + n^2 \cdot d) \cdot d^n)$. El término dominante del backtracking sigue siendo d^n , pero MRV selecciona primero las variables más restringidas y LCV preserva la flexibilidad de los dominios restantes, reduciendo sustancialmente el número de nodos explorados en la práctica.

Complejidad en espacio (Notación O)

Estructura	Espacio	Justificación
assignment = {}	$O(n)$	Diccionario con como máximo n pares variable-valor.
domains = {...} inicial	$O(n \cdot d)$	Copia local de los dominios: n variables × d valores.
saved_domains por nivel	$O(n \cdot d)$	Copia de dominios en cada nivel de recursión.
valid_values en LCV	$O(d)$	Lista temporal de valores filtrados y ordenados por LCV.
Total (n niveles activos)	$O(n^2 \cdot d)$	n niveles con $O(n \cdot d)$ cada uno, igual que Forward Checking.

La complejidad espacial es $O(n^2 \cdot d)$, idéntica a Forward Checking. MRV y LCV no requieren estructuras adicionales significativas.

5. Reducción del espacio de búsqueda vs. búsqueda exhaustiva

Una búsqueda exhaustiva evaluaría las d^n asignaciones completas sin ninguna poda. La siguiente tabla presenta los resultados experimentales obtenidos ejecutando los cuatro algoritmos sobre los diez layouts probados, midiendo el número de asignaciones intentadas y los backtracks realizados:

Layout	Algoritmo	n	d	d^n	Asign.	BT	Reducción
twin_bases	BT	2	2	4	3	1	3 / 4 (75.0%)
	FC	2	2	4	3	1	3 / 4 (75.0%)
	AC3	2	2	4	3	1	3 / 4 (75.0%)
	MRV+LCV	2	2	4	3	1	3 / 4 (75.0%)
tight_windows	BT	3	2	8	3	0	3 / 8 (37.5%)
	FC	3	2	8	3	0	3 / 8 (37.5%)
	AC3	3	2	8	3	0	3 / 8 (37.5%)
	MRV+LCV	3	2	8	3	0	3 / 8 (37.5%)

maze_csp	BT	2	2	4	2	0	2 / 4 (50.0%)
	FC	2	2	4	2	0	2 / 4 (50.0%)
	AC3	2	2	4	2	0	2 / 4 (50.0%)
	MRV+LCV	2	2	4	2	0	2 / 4 (50.0%)
heavy_cargo	BT	3	2	8	3	0	3 / 8 (37.5%)
	FC	3	2	8	3	0	3 / 8 (37.5%)
	AC3	3	2	8	3	0	3 / 8 (37.5%)
	MRV+LCV	3	2	8	3	0	3 / 8 (37.5%)
weight_distribution	BT	3	3	27	3	0	3 / 27 (11.1%)
	FC	3	3	27	3	0	3 / 27 (11.1%)
	AC3	3	3	27	3	0	3 / 27 (11.1%)
	MRV+LCV	3	3	27	3	0	3 / 27 (11.1%)
tight_overload	BT	4	2	16	4	0	4 / 16 (25.0%)
	FC	4	2	16	4	0	4 / 16 (25.0%)
	AC3	4	2	16	4	0	4 / 16 (25.0%)
	MRV+LCV	4	2	16	4	0	4 / 16 (25.0%)
storm_corridor	BT	2	2	4	2	0	2 / 4 (50.0%)
	FC	2	2	4	2	0	2 / 4 (50.0%)
	AC3	2	2	4	2	0	2 / 4 (50.0%)
	MRV+LCV	2	2	4	2	0	2 / 4 (50.0%)
storm_shortcut	BT	2	2	4	2	0	2 / 4 (50.0%)
	FC	2	2	4	2	0	2 / 4 (50.0%)
	AC3	2	2	4	2	0	2 / 4 (50.0%)
	MRV+LCV	2	2	4	2	0	2 / 4 (50.0%)

Leyenda: n = entregas, d = drones, d^n = espacio exhaustivo, Asign. = asignaciones intentadas, BT = backtracks. Verde = mayor reducción porcentual observada.

Los cuatro algoritmos tienen un comportamiento idéntico en la mayoría de los layouts: realizan el mismo número de asignaciones y backtracks. Esto refleja que en estos problemas las restricciones son suficientemente precisas para que la primera rama explorada conduzca directamente a la solución sin necesidad de retroceder.

El layout con mayor reducción porcentual es **weight_distribution** (3 drones, 3 entregas, $d^n = 27$): todos los algoritmos encuentran la solución en solo 3 asignaciones, explorando el 11.1% del espacio exhaustivo. Las restricciones de capacidad determinan unívocamente qué drone puede cubrir cada entrega, haciendo que el primer valor probado para cada variable sea el correcto.

El único layout con backtracking es **twin_bases** ($d^n = 4$), con exactamente 1 backtrack en todos los algoritmos. El primer drone probado para E1 viola la restricción de ventana de tiempo, por lo que el algoritmo retrocede y asigna el drone correcto. Que los cuatro algoritmos produzcan el mismo resultado confirma que en estos tamaños de problema las restricciones actúan como un filtro determinístico: no hay diferencia práctica entre backtracking básico y los algoritmos más sofisticados.

6. Resumen de complejidades

Algoritmo	Tiempo	Espacio	Observación
Backtracking básico	$O(n \cdot d^n)$	$O(n)$	Sin estructuras auxiliares. Mínimo uso de memoria.
Forward Checking	$O(n^2 \cdot d \cdot d^n)$	$O(n^2 \cdot d)$	Copia de dominios por nivel. Poda anticipada efectiva.
AC-3 + Backtracking	$O(n^3 \cdot d^3 \cdot d^n)$	$O(n^2 \cdot d)$	Mayor costo por nodo, mayor poda. Útil con dominios grandes.
MRV + LCV + FC	$O(n \cdot d^2 \cdot \log d \cdot d^n)$	$O(n^2 \cdot d)$	Guía la búsqueda inteligentemente. Mismo espacio que FC.

Los cuatro algoritmos son **completos**: garantizan encontrar solución si existe, ya que el espacio es finito y se evita explorar el mismo nodo dos veces. Ninguno es **óptimo** en el sentido de minimizar el costo total de la asignación: retornan la primera solución válida encontrada, no necesariamente la de menor costo de ruta.

Punto 4b

b) Búsqueda adversaria (6%)

Complejidad en tiempo y espacio de Minimax

- d = profundidad (número de plies completos; en cada ply mueve el dron y luego cada cazador).
- b = factor de ramificación (máximo de acciones legales por agente en un estado; en el grid 4-conectado $b \leq 4$).
- k = número de cazadores; hay $1 + k$ agentes en total.

Llamada inicial en `get_action`

Código	Complejidad	Explicación
num_agents = state.get_num_agents()	$O(1)$	Inicialización; lectura del número de agentes.
actions = state.get_legal_actions(0)	$O(b)$	En el peor caso el dron tiene hasta b acciones legales.
if not actions: return None	$O(1)$	Comprobación y retorno.
non_stop = [a for a in actions if a != Directions.STOP]	$O(b)$	Recorre las acciones del dron; en el peor caso b elementos.
best_action = None / best_value = float("-inf")	$O(1)$	Inicialización de variables.
for action in actions:	$O(b)$ iteraciones	En la raíz se iteran las b acciones del dron.
succ = state.generate_successor(0, action)	$O(1)$	Generar el estado sucesor (copia y actualización de posición).
value = minimax(succ, self.depth, 1)	$O(b^d)$ por hijo	Llamada recursiva que explora todo el subárbol desde ese sucesor. Hay b hijos en la raíz, cada uno con un subárbol de profundidad d y ramificación b , luego el coste total queda dominado por el árbol completo.
if value > best_value: ... best_action = action	$O(1)$	Comparación y asignación.
return best_action	$O(1)$	Retorno del resultado.

El coste de la raíz es $O(b)$ más b veces el coste del subárbol de profundidad $d - 1$ (en niveles). Eso lleva a un tiempo total $O(b^d)$.

Dentro de la función recursiva minimax(state, depth, agent_index)

Código	Complejidad	Explicación
if state.is_win() or state.is_lose() or depth == 0:	$O(1)$	Comprobaciones en tiempo constante.
return self.evaluation_function(state)	$O(1)$	Evaluación en hoja/terminal; se considera $O(1)$ o acotada.
actions = state.get_legal_actions(agent_index)	$O(b)$	En el peor caso hay hasta b acciones legales para el agente actual.

Código	Complejidad	Explicación
if not actions: return self.evaluation_function(state)	$O(1)$	Caso sin acciones; retorno con evaluación.
next_agent = (agent_index + 1) % num_agents	$O(1)$	Cálculo del siguiente agente (dron $\rightarrow$ cazador 1 $\rightarrow$... $\rightarrow$ cazador $k \rightarrow$ dron).
next_depth = depth - 1 if next_agent == 0 else depth	$O(1)$	La profundidad se decrementa solo cuando se vuelve al dron (fin de un ply completo).
if agent_index == 0: ... value = float("-inf")	$O(1)$	Nodo MAX: inicialización.
else: ... value = float("inf")	$O(1)$	Nodo MIN: inicialización.
for action in actions:	$O(b)$ iteraciones	En el peor caso se recorren las b acciones del agente.
succ = state.generate_successor(agent_index, action)	$O(1)$	Generar un sucesor; copia del estado y cambio de posición.
value = max(value, minimax(succ, next_depth, next_agent)) (MAX)	recursivo	Llamada recursiva; el subárbol tiene un nivel menos de profundidad efectiva.
value = min(value, minimax(succ, next_depth, next_agent)) (MIN)	recursivo	Idem para nodos MIN.
return value	$O(1)$	Retorno del valor minimax del nodo.

Por cada nodo no terminal se hace $O(b)$ trabajo (obtener acciones y bucle) más b llamadas recursivas. En el peor caso se visitan todos los nodos del árbol.

Tamaño del árbol: En cada ply hay 1 nivel MAX (dron) y k niveles MIN (cazadores), es decir $1 + k$ niveles por ply. La profundidad en *niveles* es $d \cdot (1 + k)$. En cada nivel cada nodo tiene hasta b hijos, así que el número de nodos es del orden de $b^{(d \cdot (1+k))}$. En notación simplificada, con d como profundidad efectiva en plies: **número de nodos = $O(b^d)$** .

3. Conclusión: tiempo y espacio en función de d y b

Tiempo: Cada nodo realiza $O(b)$ operaciones (obtener acciones, bucle sobre acciones, generar sucesores, comparar valores). En el peor caso se expanden todos los nodos del árbol, que son $O(b^d)$. Por tanto:

- **Complejidad en tiempo: $O(b^d)$** en función de la profundidad d y el factor de ramificación b .

Espacio: La pila de recursión tiene un marco por cada nivel del árbol. La profundidad del árbol en niveles es $O(d)$ (plies completos). En cada marco se guardan las variables locales (*state*, *depth*, *agent_index*, *actions*, *value*, etc.); si el estado se pasa por referencia, el espacio por marco es $O(1)$. Por tanto:

- **Complejidad en espacio: $O(d)$** en función de la profundidad d .

pesimista con cazadores no óptimos

Minimax asume que el adversario (MIN) juega óptimamente en todo momento: siempre elige la acción que minimiza el valor para el dron. Por tanto, el valor minimax es el peor caso ante un adversario perfecto.

Si los cazadores no son óptimos (por ejemplo, se mueven de forma aleatoria o con errores):

- Minimax sigue planificando como si fueran óptimos, es decir, sobrestima la amenaza.
- El dron puede rechazar líneas de juego que en la práctica serían ganadoras porque un cazador real no las explotaría.
- En resumen: Minimax es pesimista porque asume el peor comportamiento posible del cazador. En entornos con cazadores estocásticos o subóptimos, Expectimax (o un modelo mixto) es más adecuado porque modela la incertidumbre y suele llevar a mejores tasas de victoria en la práctica.

Función de evaluación: características, pesos y justificación

Características del estado usadas:

1. Distancia del dron a la entrega más cercana (Dijkstra con coste por terreno: libre, niebla, montaña, tormenta).
 - a. Peso: $+20 / (\text{distancia} + 1)$.
 - b. Motivo: Acercarse a una entrega es el objetivo principal; Dijkstra refleja el coste real de batería y evita terrenos caros cuando no compensan.
2. Distancia de cada cazador al dron (BFS con `hunter\_restricted=True` : solo terreno libre).
 - a. Pesos:
 - i. Distancia ≤ 1 : -300 (riesgo de captura inminente).
 - ii. Distancia ≤ 2 : -80 .
 - iii. Distancia ≤ 3 : -20 .
 - iv. Distancia > 3 : $+0.1 \times \text{distancia}$ (ligero bonus por estar lejos).
 - b. Si la distancia es infinita (cazador no puede alcanzar al dron), no se penaliza.
 - c. Motivo: Los cazadores solo caminan por celdas libres; en niebla/montaña/tormenta el dron puede ganar ventaja. Las penalizaciones fuertes cerca evitan arriesgarse; el pequeño bonus por distancia anima a mantener separación.
3. Número de entregas pendientes.
 - a. Peso: -50 por cada entrega pendiente.
 - b. Motivo: Priorizar terminar la misión y no quedarse solo evadiendo sin avanzar.
4. Puntuación actual del juego (`state.get\_score()`).
 - a. Base que ya incluye penalizaciones por tiempo y recompensas por entregas.

Resumen: La función equilibra “acercarse a entregas” y “evitar cazadores”, con pesos mayores para evitar la captura cuando el cazador está muy cerca.

Comparación de tasas de victoria en diversos layouts

Comparación cualitativa esperada:

- Minimax vs AlphaBeta: Mismas decisiones y mismas tasas de victoria; AlphaBeta es más rápido al podar ramas.
- Layouts con cazadores cercanos (ej. contested_territory, gauntlet): Minimax/AlphaBeta con profundidad suficiente pueden ganar más que RandomAgent; la función de evaluación influye mucho.
- Layouts con terreno favorable al dron (tunnel_escape, mountain_overfly): Tasas de victoria altas para todos los agentes que usen bien la evaluación.
- Cazadores estocásticos ($p > 0$): Expectimax con p calibrado (ej. 0.2–0.4) suele superar a Minimax en tasa de victorias porque no asume que el cazador siempre juega óptimo.

Layout	Minimax (d=2)	AlphaBeta (d=2)	Expectimax (d=2, p=0.3)
tiny_hunt	20/20	20/20	10/10
small_hunt	10/10	10/10	10/10
contested_territory	10/10	10/10	10/10

Nodos adicionales que poda AlfaBeta (mejor y peor caso)

- Mejor caso (orden óptimo de exploración): Los mejores movimientos se exploran primero. En un árbol con profundidad d y ramificación b , el número de nodos explorados puede reducirse a $O(b^{(d/2)})$ (solo se expande “un lado” del árbol en cada nivel cuando la poda es máxima). Por tanto, los nodos podados son del orden de $O(b^d - b^{(d/2)})$.
- Peor caso (orden pésimo de exploración): Si siempre se exploran primero las ramas que no permiten podar (por ejemplo, el peor movimiento primero en MIN y el mejor último), no se poda nada. Nodos podados = 0.

En la práctica, con ordenación heurística o aleatoria, AlfaBeta suele podar una fracción importante de nodos (por ejemplo, entre un tercio y la mitad en muchos juegos), lo que se puede comprobar instrumentando el código para contar nodos expandidos en Minimax vs AlphaBeta con el mismo layout y profundidad.

c) Reflexión sobre la coconstrucción con IA (3%)

Utilizamos la IA como apoyo en varias partes del taller. Inicialmente intentamos resolver cada punto de manera autónoma (backtracking, Minimax, Alpha-Beta, Expectimax y la función de evaluación). Cuando encontrábamos errores de lógica o de estructura, compartíamos nuestro código y el enunciado del taller para solicitar correcciones específicas.

Uno de los principales aprendizajes que logramos tener surgió al analizar las correcciones sugeridas por la IA. En el caso de backtracking, nos dimos cuenta de la importancia de seleccionar una única variable no asignada y devolver correctamente una copia de la asignación al retornar. En Minimax, aclaramos cómo manejar la profundidad del árbol y cuándo pasar al siguiente agente. En Alpha-Beta, entendimos mejor las condiciones de poda y el orden en que deben actualizarse los valores *alpha* y *beta*. En Expectimax, vimos por qué los nodos de azar no permiten poda y cómo $(1 - p) \cdot \min + p \cdot \text{promedio}$ puede modelar comportamientos mixtos de los cazadores. Finalmente, en la función de evaluación

aprendimos a combinar heurísticas como Dijkstra para estimar distancias a entregas, BFS con restricciones de terreno para los cazadores y pesos que equilibran la prioridad entre completar entregas y evitar capturas.

La construcción de prompts útiles no fue tan sencilla y honestamente consideramos que es un proceso iterativo y que requiere de mucho análisis previo y entendimiento del problema. Para obtener sugerencias realmente aplicables fue necesario incluir suficiente contexto del problema (roles de los agentes como dron MAX y cazadores MIN o CHANCE, turnos completos, profundidad del árbol) y compartir fragmentos de nuestro código. En varios casos tuvimos que iterar hasta obtener una solución que respetara el enunciado y superara las pruebas.

En términos de comprensión, las soluciones sugeridas fueron “buenas” una vez lográramos revisarlas y contrastarlas con nuestro trabajo previo. Esto nos permitió comprender las decisiones algorítmicas y considerar que podríamos usar las soluciones ya que teníamos un “conocimiento” que nos permitió aprobar las decisiones y entender por qué las mejoras eran realmente una “mejora”, apoyándonos tanto en el código como en los conceptos teóricos.

En conclusión, la IA fue una herramienta útil para superar bloqueos puntuales, pero el aprendizaje real surgió al analizar críticamente cada sugerencia, compararla con el enunciado y el contenido del curso, y validar cada cambio antes de incorporarlo. De esta forma mantuvimos el control del desarrollo y aseguramos comprender la solución final.